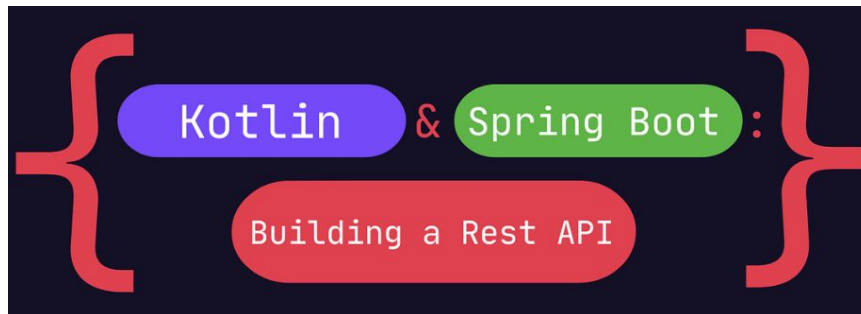


Desarrollo de Sistemas Móviles

Sesión 2: Frameworks de Desarrollo Móvil y Fundamentos de Kotlin

Al finalizar la clase, serás capaz de:

- Comparar frameworks móviles modernos.
- Comprender por qué Kotlin domina Android.
- Aplicar conceptos fundamentales de Kotlin.
- Entender Jetpack Compose y estado reactivo.
- Reconocer la arquitectura moderna Android.
- experimentar una pincelada de Dart.



Framework	Lenguaje	Plataformas	Ideal para
Kotlin + Compose	Kotlin	Android	Apps premium
Flutter 4.x	Dart	Android/iOS/Web /Desktop	Multiplataforma
React Native	TypeScript	Android/iOS	Equipos web
Kotlin Multiplatform	Kotlin	Android + iOS	Lógica compartida
.NET MAUI	C#	Android/iOS/Desktop	Ecosistema Microsoft

Tendencias en 2026

- Kotlin domina Android.
- Flutter domina multiplataforma.
- React Native sigue fuerte en startups.
- KMM crece en enterprise.

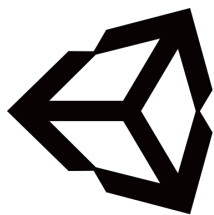
2026

Recomendadas en 2026

Android



- Kotlin 2.0+
- **Jetpack Compose**
- StateFlow
- **Coroutines**
- MVVM + Clean
- KMM



unity

- videojuegos,
- realidad aumentada,
- experiencias inmersivas.



Tecnologías en desuso

- Xamarin → migrar a .NET MAUI
- Java legacy pre-Java 21
- MVC
- MVP
- LiveData
- AsyncTask
- XML complejo + Fragment hell

Recomendadas en 2026

- Flutter 4.x & Dart 3.x
- Riverpod
- Declarative UI
- Clean Architecture
- Freezed + sealed unions
- GoRouter
- Async/Await + Streams



Kotlin se convirtió en el estándar Android porque combina productividad, seguridad y rendimiento nativo.

Actualmente es el lenguaje principal recomendado por Google para aplicaciones Android modernas.

Kotlin es el estándar Android moderno

Ventajas principales

- Lenguaje oficial Android.
- Más seguro que Java.
- Menos código boilerplate.
- Null safety integrado.
- Coroutines nativas.
- Excelente interoperabilidad con Java.

Kotlin 2.0+

- Compilador K2.
- Builds más rápidos.
- Mejor inferencia de tipos.
- Kotlin Multiplatform estable..

Variables y tipado

Immutable

```
val nombre = "Android"
```

```
var contador = 0
```

Mutable

Kotlin utiliza un sistema de tipado estático moderno que permite escribir código más seguro y legible.

Una de sus principales características es diferenciar entre variables mutables e inmutables.

NULL Safety

Uno de los problemas históricos más comunes en Java era el famoso **NullPointerException**.

Kotlin incorpora un sistema de **manejo de NULLs** que ayuda a prevenir este tipo de errores desde el compilador.

```
var nombre: String = "Kotlin"  
var apodo: String? = null
```

```
println(apodo?.length)
```

Funciones y lambdas

Las **funciones** permiten dividir el código en bloques reutilizables y organizados.

Kotlin además trata a las funciones como ciudadanos VIP, permitiendo usar **lambdas** y **programación funcional** de manera muy sencilla.

```
fun saludar(nombre: String): String {  
    return "Hola $nombre"  
}
```

```
fun cuadrado(n: Int) = n * n
```

Expression Body Function

Single-Expression Function

Funciones y lambdas

Las **lambdas** son funciones anónimas muy utilizadas en Android moderno, especialmente con **Compose**, **Flow** y **colecciones**.

```
val suma = { a: Int, b: Int -> a + b }
```

```
val numeros = listOf(1, 2, 3, 4, 5)
```

```
val pares = numeros.filter { it % 2 == 0 }
```

```
val dobles = numeros.map { it * 2 }
```

Estas operaciones permiten transformar listas de forma declarativa y legible.



Kotlin genera automáticamente métodos esenciales para comparar, copiar y mostrar objetos, reduciendo boilerplate y mejorando la legibilidad del código.

```
data class Producto(  
    val id: Int,  
    val nombre: String,  
    val precio: Double  
)
```

Funciones automáticas

- equals()
- hashCode()
- toString()
- copy()

Kotlin genera automáticamente métodos esenciales para comparar, copiar y mostrar objetos, reduciendo boilerplate y mejorando la legibilidad del código.

```
data class Producto(  
    val id: Int,  
    val nombre: String,  
    val precio: Double  
)
```

Funciones automáticas

- equals()
- hashCode()
- toString()
- copy()

Las **sealed classes** permiten representar estados de interfaz de manera segura y controlada.

- Android moderno
- modelar pantallas (carga, éxito o error),
- manejo del estado reactivo en Compose y MVVM

```
sealed class UiState {  
    data object Loading : UiState()  
    data class Success(val data: String) : UiState()  
    data class Error(val message: String) : UiState()  
}
```

Class Type	Key Points	When to Use
Regular Class	- Standard template for creating objects with properties and functions.	- For general templates with both properties and functions.
Abstract Class	- Used as a blueprint; cannot be directly instantiated. - Serves as a base for more specific classes.	- To provide a common base for other classes with shared behaviors.
Interface	- Defines a set of methods that classes must implement. - Supports multiple interface implementation by a class.	- When defining a contract that multiple classes should implement.
Enum Class	- Represents a set of constants with individual instances. - Useful for defining and working with predefined options.	- When you have a fixed set of constants representing distinct options.
Data Class	- Specifically designed for holding data. - Automatically generates useful methods for data comparison and string representation.	- Specifically for holding data without additional behavior.
Sealed Class	- Restricts subclasses to a predefined set. - Enhances code predictability and structure.	- To restrict subclasses to a predefined set, enhancing code predictability.
Companion Object	- Contains static members (properties or functions) associated with a class. - Shared among all instances of the class.	- When you need static members associated with a class.
Inline Class (Value Class)	- Wraps a single value for type safety and efficiency. - Introduced with the inline keyword.	- To wrap a single value for type safety and efficiency.

✓ Choose a class type that aligns with the nature of the problem you are solving. Prioritize clarity and maintainability over complexity.

✓ Be mindful of the relationships between classes, especially when using inheritance or interfaces.

✓ Use enums for a predefined set of options, data classes for data storage, and sealed classes for a closed set of subclasses.

www.pinartechtips.com



Las aplicaciones móviles modernas realizan muchas tareas al mismo tiempo: consumir APIs, descargar imágenes, acceder a bases de datos y actualizar interfaces.

Si estas operaciones **se ejecutan en el hilo principal**, la aplicación puede congelarse.



En Android, la UI funciona en un único hilo principal (**Main Thread**).

Si una operación pesada bloquea ese hilo:

- la aplicación se congela,
- aparecen errores ANR (*Application Not Responding*),
- y la experiencia del usuario empeora.

Las coroutines permiten mover esas tareas a segundo plano de forma segura y sencilla.

Las **coroutines** simplifican enormemente el trabajo asíncrono en Android.

Permiten escribir código **concurrente** de forma secuencial, legible y mucho más fácil de mantener.

Antes

- AsyncTask
- Callbacks anidados
- RxJava complejo

```
api.getUser(object : Callback {  
    override fun onSuccess(user: User) {  
        api.getPosts(user.id)  
    }  
})
```

Ahora

- Coroutines
- Structured Concurrency
- Flow

```
suspend fun cargarDatos() {  
    val user = api.getUser()  
    val posts = api.getPosts()  
}
```


Las aplicaciones modernas son **reactivas**: la interfaz cambia automáticamente cuando cambia el estado. **Flow** y **StateFlow** son actualmente la solución oficial basada en coroutines para manejar estados y eventos.

LiveData

Legacy

Android only

Menos flexible

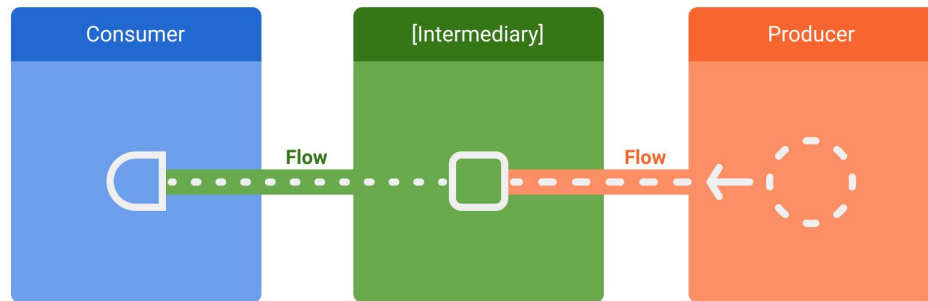
StateFlow

Moderno

Kotlin multiplatform

Coroutines-first

```
flow {  
    emit("Hola")  
}
```



- **Flow** es una API de Kotlin basada en Coroutines que permite emitir datos de manera asíncrona y continua.
- Y un **StateFlow** mantiene un estado observable y persistente.



Jetpack Compose representa el cambio más importante en el desarrollo Android moderno.

En lugar de construir interfaces usando archivos **XML separados**, la UI ahora se crea completamente con Kotlin.

Compose utiliza un **enfoque declarativo**: la interfaz se describe en función del estado actual de la aplicación.

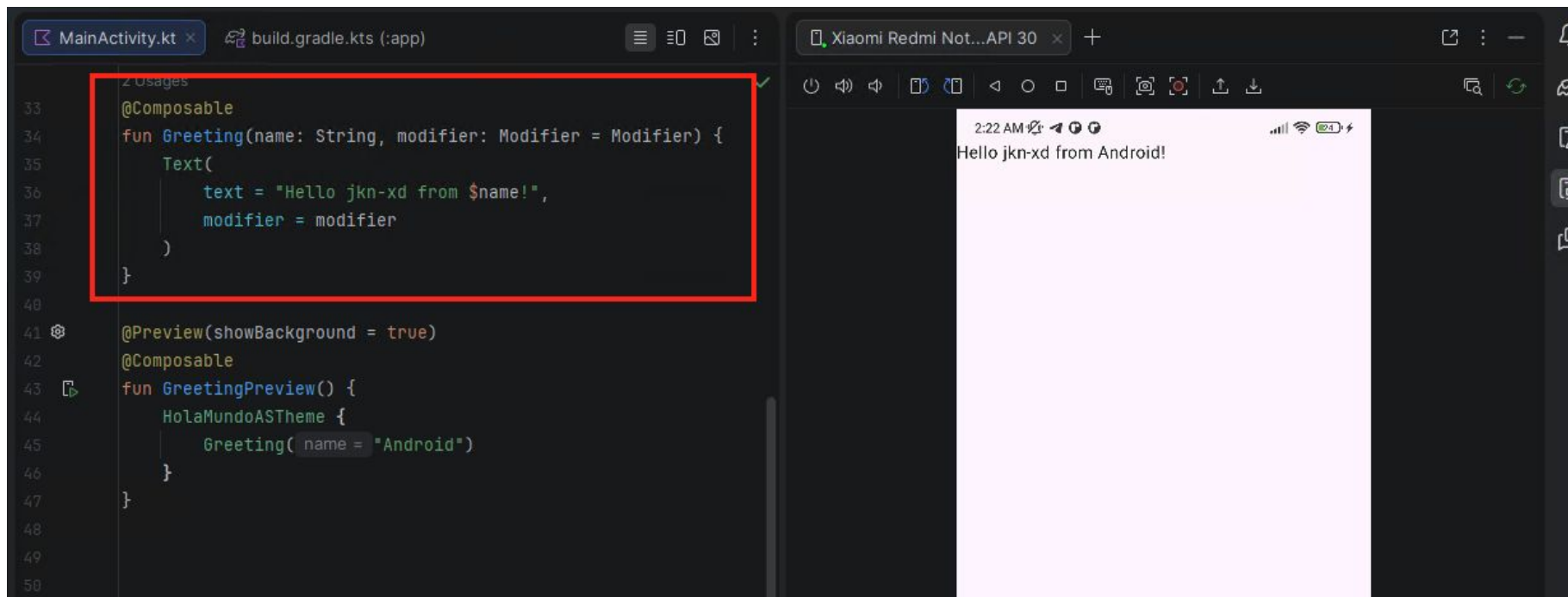
Antes

- Mucho boilerplate.
- Mayor complejidad.
- UI difícil de mantener.
- Código distribuido entre XML y Kotlin.

Ahora

- UI declarativa
- Estado reactivo
- Kotlin puro
- Recomposición automática

Las funciones anotadas con `@Composable` representan componentes visuales reutilizables.



Estado reactivo

```
var contador by remember {  
    mutableStateOf(0)  
}
```

Un modelo de estado reactivo implica que la interfaz se actualice automáticamente cada vez que cambia la información.

Novedades modernas (compose 1.7+)

- Performance optimizations
- Predictive recomposition
- Runtime profiling tools
- Adaptive Layout system
- Mejor soporte foldables/tablets

Eventos

```
Button(onClick = {  
    contador++  
}) {  
    Text("Incrementar")  
}
```

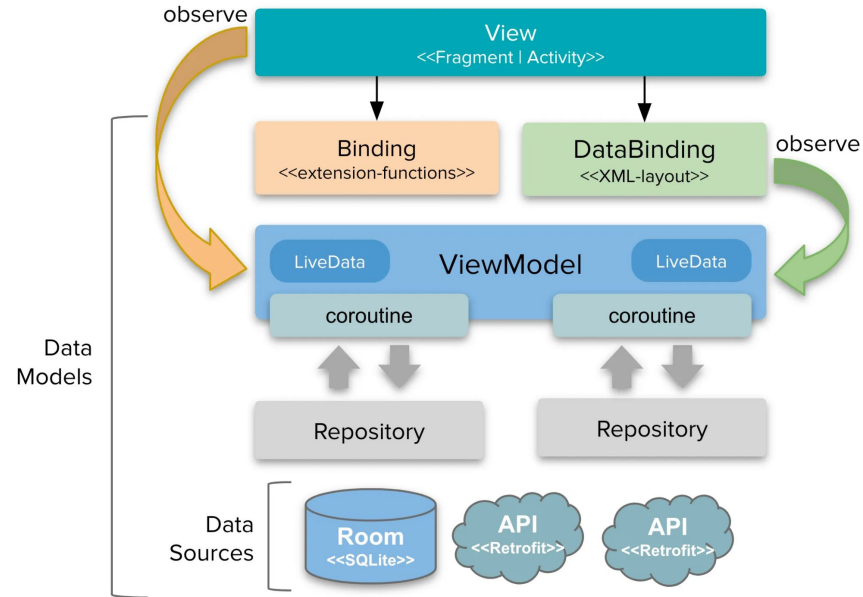
Los eventos del usuario, como clics o cambios de texto, modifican el estado de la aplicación y provocan la recomposición automática de la UI. Enfoque moderno: patrón UDF (Unidirectional Data Flow), popular en Android 2026.



Las aplicaciones Android modernas pueden crecer rápidamente en tamaño y complejidad. Sin una arquitectura clara, es común terminar con **Activities gigantes**, **lógica duplicada** y **aplicaciones difíciles de mantener**.

MVVM y Clean Architecture ayudan a organizar el proyecto separando responsabilidades.

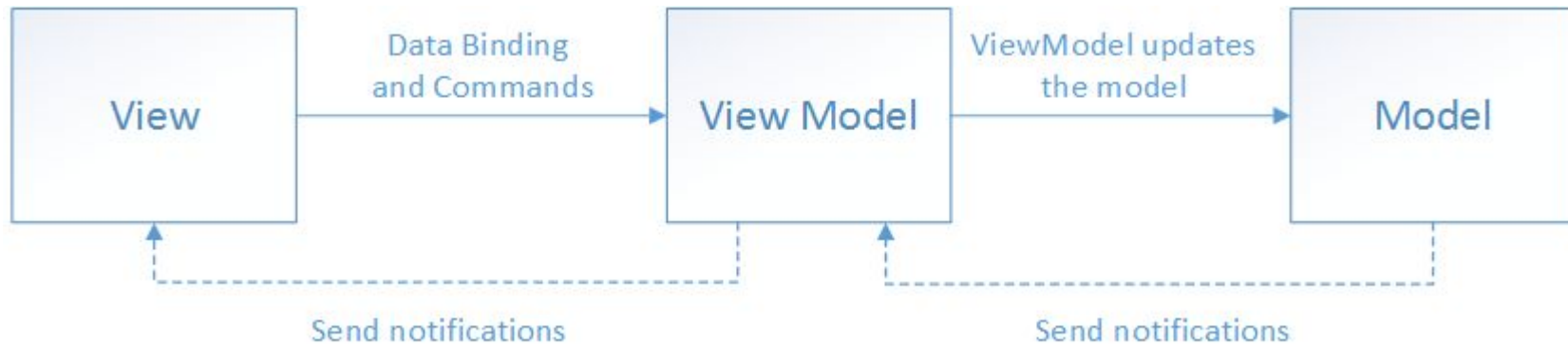
Capa	Responsabilidad
UI Layer	Contiene Compose Screens y componentes visuales
ViewModel	Gestiona estados y lógica de presentación
UseCase	Representa reglas de negocio específicas
Repository	Coordina datos locales y remotos
DataSource	Conexión con APIs o bases de datos

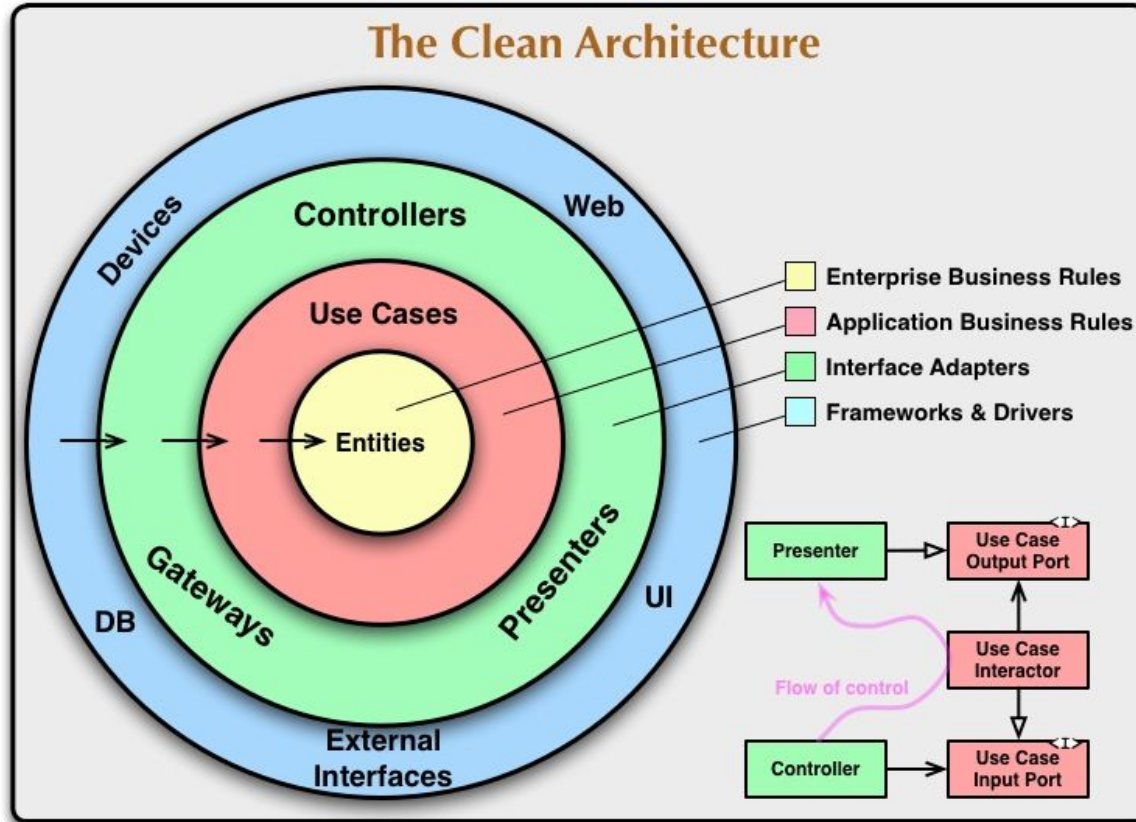


Arquitectura	Características	Problemas principales
MVC	La vista y lógica suelen mezclarse	Activities muy grandes (Massive Activities)
MVP	Usa Presenter para separar lógica	Mucho boilerplate y complejidad
MVVM	Usa ViewModel y estado reactivo	Más simple y escalable

El patrón MVVM ayuda a separar limpiamente la lógica de presentación y negocios de una aplicación de su interfaz de usuario (UI).

MVVM





Clean Architecture es un estilo de arquitectura de software que organiza la aplicación en capas separadas para que el código sea más mantenible, escalable y fácil de probar.

Su objetivo principal es:

- Separar la lógica de negocio de la UI, frameworks y bases de datos.

Clean Architecture está estructurada en tres capas principales, cada una con un rol específico:

- Capa de Presentación: Maneja la UI y las interacciones del usuario.
- Capa de Dominio: Contiene la lógica de negocio, casos de uso e interfaces de repositorio.
- Capa de Datos: Implementa los repositorios, maneja llamadas a APIs y operaciones de base de datos.

El principio central de **Clean Architecture** es la dirección de las dependencias: cada capa solo debe depender de las capas más cercanas al núcleo (**dominio**). Esto garantiza flexibilidad y escalabilidad.

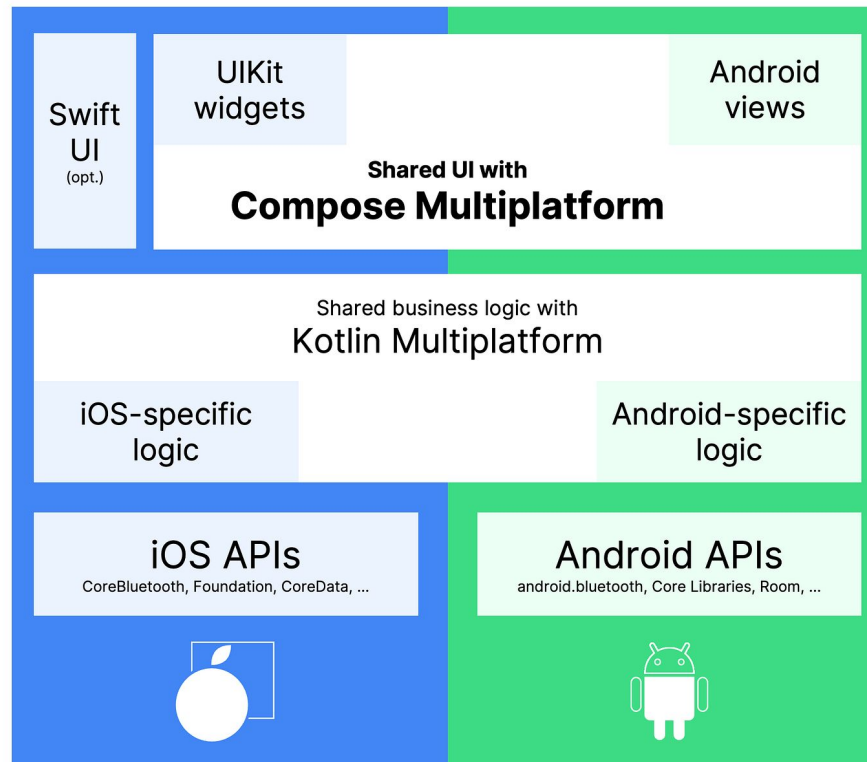
Code

```
com.example.app
|— app (Módulo principal de la aplicación)
|— feature-user
|   |— domain (Casos de Uso, Interfaces de Repositorio, Modelos)
|   |— data (Implementaciones de Repositorio, Fuentes de Datos, APIs, DB)
|   |— presentation (UI y ViewModels para funcionalidades de usuario)
|— core (Utilidades comunes, networking, helpers de base de datos)
```



Kotlin Multiplatform permite compartir lógica entre Android e iOS sin perder interfaces nativas. Esto reduce costos de desarrollo y facilita mantener aplicaciones en múltiples plataformas.

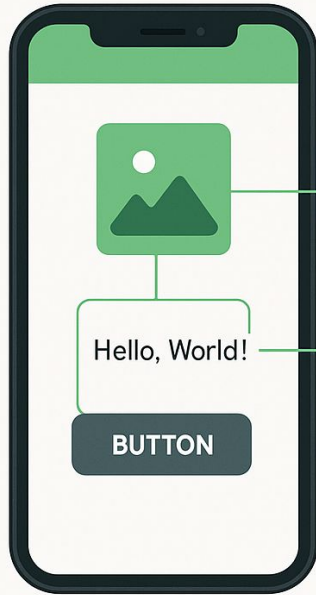
- lógica de negocio
- networking
- persistencia
- validaciones
- Compose Android (Mantiene UI nativa)
- SwiftUI iOS (Mantiene UI nativa)



	Kotlin Multiplatform	Flutter
Created by / Language	JetBrains / Kotlin	Google / Dart
Flexibility and code reuse	Share whichever part of your codebase you want to, including business logic and/or UI, from 1% to 100%.	Control every pixel of an application to create customized and adaptive designs with 100% code sharing across all platforms.
Packages, dependencies, and ecosystem	Packages are available from Maven Central and other repositories, including klibs.io (Alpha version), which is designed to simplify the search for KMP libraries. This list contains some of the most popular KMP libraries and tools.	Packages are available from Pub.dev .
Build tool	Gradle (plus Xcode for applications targeting Apple devices).	The Flutter command line tool (which uses Gradle and Xcode under the hood).
Code sharing	Android, iOS, web, desktop, and server-side.	Android, iOS, web, desktop, and embedded devices.
Compilation	Compiles to JVM bytecode for desktop and Android, JavaScript or Wasm on the web, and platform-specific binaries for native platforms	Debug builds run Dart code in a virtual machine. Release builds output platform-specific binaries for native platforms, and JavaScript/Wasm for the web.
Communication with native APIs	Native APIs are accessible directly from Kotlin code using expect/actual declarations .	Communication with the host platform is possible using platform channels .
UI rendering	Compose Multiplatform can be used to share UI across platforms, based on Jetpack Compose by Google, using the Skia engine that is compatible with OpenGL, ANGLE (which translates OpenGL ES 2 or 3 calls to native APIs), Vulkan, and Metal. UI previews are available even from common code.	Flutter widgets are rendered on the screen using the custom Impeller engine , which talks straight to the GPU using Metal, Vulkan, or OpenGL depending on the platform and device.
Iteration on UI development	With Compose Hot Reload , you can instantly see UI changes without restarting your app or losing its state.	IDE plugins are available for VS Code and Android Studio.
Companies using the technology	Forbes , Todoist , McDonald's , Google Workspace , Philips , 9gag , Baidu , Autodesk , TouchLab , Instabee , and more are listed in our KMP case studies .	Xiaomi , Wolt , Universal Studios , Alibaba Group , ByteDance , Geico , eBay Motors , Google Pay , So Vegan , and more are listed on the Flutter Showcase .

XML Layout in Android Explained

BEGINNER
FRIENDLY



activity_main.xml

```
<LinearLayout>
  xmlns:android="http://
  schemas.android.com/apk/
  res/android"
  android:orientation="vertical"
</LinearLayout>

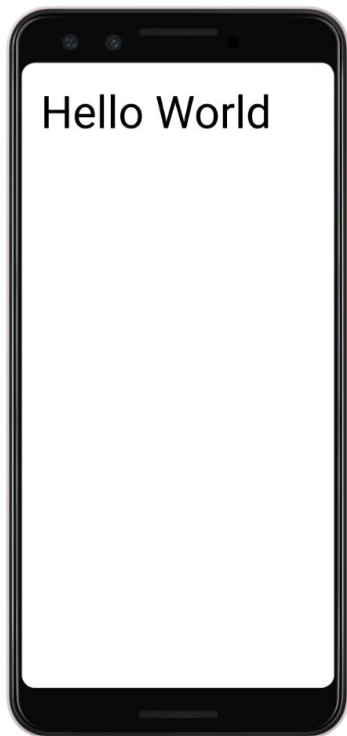
<ImageView>

</ImageView>

<TextView>
  android:layout_width="wrap
  content"
  android:layout_height="wrap
  content"
</TextView>

<Button>
```

BEGINNER FRIENDLY



```
@Composable
fun Greeting(name: String) {
    Text("Hello $name")
}
```



Lab

https://github.com/johnkbarrera/johnkbarrera.github.io/blob/main/clases/mobile/mobile_lab2.pdf



